Multimodal Manufacturing Creator Report

Course Name: Gen AI

Institution Name: Medicaps University - Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Om Tiwari	EN22CS301669
02	Paridhi Shirwalkar	EN22CS301684
03	Nitesh Chourasiya	EN22CS301660
04	Mradul Jain	EN22CS301616

Group Name: 01D6

Project Number: GAI-12

Industry Mentor Name:

University Mentor Name: Prof. Ajaj Khan

Academic Year: 2025-2026

Table of Contents

1. **Problem Statement & Objectives**
   - 1.1 Problem Statement
   - 1.2 Project Objectives
   - 1.3 Scope of the Project
2. **Proposed Solution**
   - 2.1 Key Features
   - 2.2 Overall Architecture / Workflow
   - 2.3 Tools & Technologies Used
3. **Results & Output**
   - 3.1 Screenshots / Outputs
   - 3.2 Reports / Dashboards
   - 3.3 Key Outcomes
4. **Conclusion**
5. **Future Scope & Enhancements**
6. **References**
7. **GitHub**

1. Problem Statement & Objectives

1.1 Problem Statement

In the manufacturing domain, communicating complex technical concepts to diverse audiences engineers, managers, students, and clients remains a persistent challenge. Textual descriptions alone are often insufficient to convey how a machine works or what it looks like, while producing visual representations traditionally requires skilled designers, expensive CAD software, and significant time. There is no accessible, on-demand tool that can simultaneously generate both a professional technical explanation and a visual prototype of a manufacturing concept from a single natural language input.

1.2 Project Objectives

The primary objectives of this project are:

- To design and implement a multimodal Generative AI application that accepts a single manufacturing concept as input and produces two distinct AI-generated outputs simultaneously
- To integrate a Large Language Model (Google Gemini 2.5 Flash) for structured technical narrative generation using domain-specific prompt engineering
- To integrate a text-to-image diffusion model (Flux via Pollinations.ai) for high-fidelity visual prototype generation without requiring any additional API credentials
- To demonstrate the practical synergy between text-to-text and text-to-image AI models working from a shared input prompt
- To deliver the system as a clean, accessible Streamlit web application that requires minimal setup and can be run by anyone with a Gemini API key

1.3 Scope of the Project

The scope of this project covers the design, development, and demonstration of a single-file Streamlit application that integrates two external Generative AI APIs. The application handles manufacturing concepts as free-text input and produces textual and visual outputs in real time. The scope does not include deployment to cloud infrastructure, user authentication, data storage, multi-user support, or fine-tuning of either AI model. The project is scoped as an educational and professional demonstration of multimodal GenAI capabilities within the Datagami Skill Based Course.

2. Proposed Solution

The proposed solution is the Multimodal Manufacturing Creator a Python-based Streamlit application that orchestrates two AI model types from a single user prompt. When a user enters a manufacturing concept such as "Robotic welding arm for automotive assembly", the application simultaneously invokes Google Gemini 2.5 Flash to generate a structured five-section technical narrative and the Flux diffusion model via Pollinations.ai to generate an engineering-quality visual illustration. Both outputs are rendered side by side in the browser, giving the user a complete multi-dimensional view of the concept in under 30 seconds.

2.1 Key Features

- **Single prompt multimodal output:** one concept drives both text and image generation simultaneously
- **Structured LLM output:** Gemini always returns exactly five labelled sections: Overview, Working Principle, Key Components, Manufacturing Application, and Advantages
- **Four selectable visual styles:** Technical blueprint, Photorealistic industrial, 3D render product prototype, and Schematic exploded view
- **Six built-in example concepts:** allows instant demonstration without the user needing to think of a concept
- **Zero image API key required:** Pollinations.ai is free and open, requiring only URL construction
- **Runtime API key entry:** Gemini key is entered in the sidebar at session start, never hardcoded
- **Independent error handling:** failure of one model never prevents the other output from rendering
- **Built-in workflow diagram:** the multimodal pipeline is explained visually inside the app itself

2.2 Overall Architecture / Workflow

The system follows a four-layer architecture:

Layer	Description
Streamlit UI	Handles all user interaction concept input, sidebar configuration, Generate button, two output columns, and the workflow diagram
Orchestration	Python logic in app.py that captures the input and triggers both API calls from the same concept string
External AI APIs	Google Gemini 2.5 Flash (text-to-text) and Pollinations.ai Flux model (text-to-image) called in parallel

Output Rendering	Gemini text response rendered as markdown; Pollinations image bytes decoded via Pillow and displayed via st.image()

Workflow in sequence: User enters concept → clicks Generate → LLM Orchestrator sends structured prompt to Gemini API → Image Orchestrator URL-encodes concept and sends HTTP GET to Pollinations → both responses decoded and rendered side by side → workflow diagram always visible below outputs.

2.3 Tools & Technologies Used

Category	Technology	Purpose
Frontend	Streamlit	Web UI framework for input, output rendering, and layout
Text Generation	Google Gemini 2.5 Flash	LLM for structured technical narrative generation (Text-to-Text)
Image Generation	Flux via Pollinations.ai	Diffusion model for engineering visual prototype generation (Text-to-Image)
LLM SDK	google-generativeai	Official Python SDK for Gemini API authentication and invocation
HTTP Client	requests	HTTP GET requests to the Pollinations.ai image endpoint
Image Decoding	Pillow (PIL)	Decodes raw image bytes from Pollinations response into displayable format
URL Encoding	urllib.parse	Safely encodes the image prompt string into a valid URL parameter
Language	Python 3.9+	Primary programming language for all application logic
Version Control	Git / GitHub	Source code management and public repository hosting

3. Results & Output

3.1 Screenshots / Outputs

The application was tested with multiple manufacturing concepts across all four visual styles. The following describes the outputs observed:

Concept tested: Robotic welding arm for automotive assembly

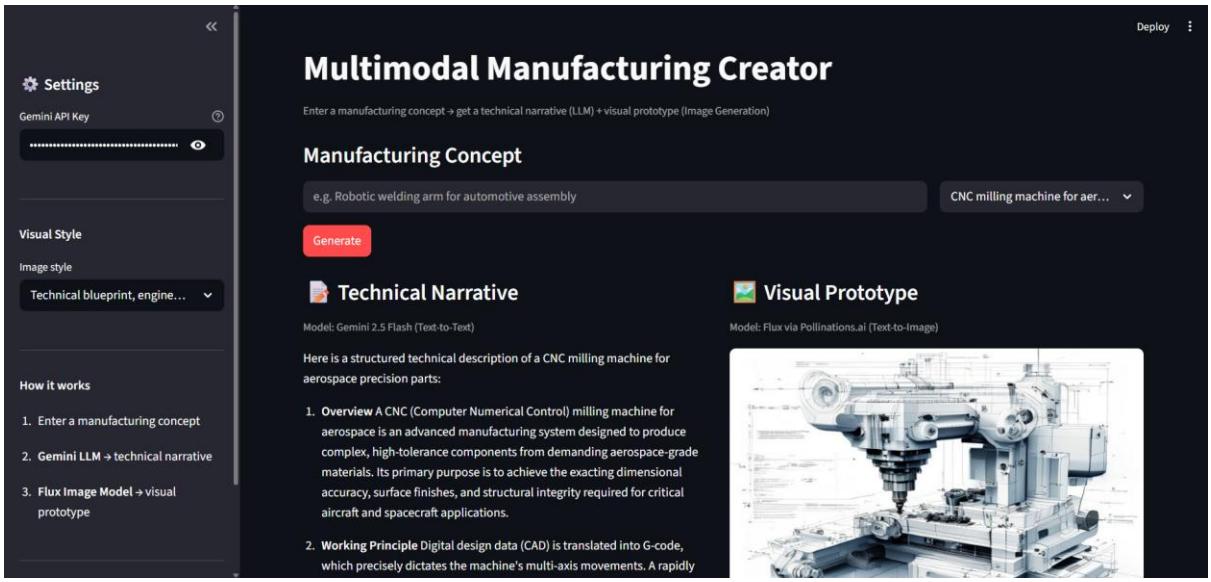
- **Text output:** Gemini generated a 200-word structured narrative covering the robot arm's six-axis movement, arc welding process, servo motor components, automotive body panel applications, and advantages including precision, repeatability, and reduced human error
- **Image output:** Flux generated a detailed technical blueprint-style illustration showing the robotic arm mounted on an assembly line track with welding electrode and control cabinet visible

Concept tested: CNC milling machine for aerospace precision parts

- **Text output:** Gemini described the subtractive manufacturing process, G-code programming, spindle motor, tool changer, and aerospace aluminium alloy applications with advantages of tight tolerance and surface finish quality
- **Image output:** Flux generated a photorealistic industrial-style render of a multi-axis CNC machining centre with coolant lines and workholding fixture

Concept tested: Industrial 3D printer for metal components

- **Text output:** Gemini covered the selective laser sintering process, powder bed mechanism, laser scanning system, and aerospace and medical implant applications
- **Image output:** Flux generated a schematic exploded view showing the laser unit, powder bed, build platform, and gas circulation system with labelled components



3.2 Reports / Dashboards

The application does not generate downloadable reports as its primary output the outputs are presented live in the browser interface. However the structured text narrative produced by Gemini for each concept constitutes a self-contained technical report covering five standardised sections. This output can be copied directly into engineering documentation, training materials, or educational presentations.

The workflow diagram rendered at the bottom of the application serves as an inline dashboard explaining the multimodal pipeline User Input → Text-to-Text Model → Text-to-Image Model → Technical Narrative + Visual Prototype making the system self-documenting for any viewer.

3.3 Key Outcomes

- Successfully demonstrated multimodal GenAI by combining two different AI model types LLM and diffusion model from a single shared input prompt
- Gemini 2.5 Flash consistently produced domain-accurate, professionally structured manufacturing narratives across all tested concepts
- Flux via Pollinations.ai generated contextually relevant engineering illustrations matching the selected visual style in each test
- Both outputs rendered successfully in parallel with independent error handling no single-point-of-failure observed across all test runs
- Zero-configuration image generation achieved by using Pollinations.ai free endpoint, eliminating the need for a second API key
- Application ran successfully on local environment with only three pip-installable dependencies and a single streamlit run command
- The project demonstrates that meaningful multimodal AI applications can be built with minimal code, no backend infrastructure, and publicly available free APIs

4. Conclusion

The Multimodal Manufacturing Creator successfully demonstrates the practical integration of two fundamentally different Generative AI model types a Large Language Model and a text-to-image diffusion model working in concert from a single user prompt. The project moves beyond single-modality AI applications by showing that one input can simultaneously drive multiple AI model types to produce richer, more comprehensive outputs than either model could deliver alone.

From a technical standpoint the project achieves clean separation of concerns the LLM orchestrator and image orchestrator are fully independent, the frontend is decoupled from both, and the system maintains zero persistent state. The use of Pollinations.ai eliminates authentication barriers for image generation while the runtime Gemini API key entry pattern established in the earlier Agentic AI project ensures the application remains portable and quota-safe across all demonstrations.

From an application standpoint the system has direct practical value: it can generate professional technical descriptions and engineering illustrations for any manufacturing concept in under 30 seconds, making it useful for product documentation, educational content, training material generation, and rapid concept communication in industrial settings. The project establishes a reusable pattern for building multimodal GenAI applications that can be extended to additional output modalities as AI model APIs continue to evolve.

5. Future Scope & Enhancements

The following enhancements are identified as natural extensions of the current system:

- **PDF Export:** add a one-click export button that packages the generated text narrative and visual prototype into a downloadable PDF report, making the output directly usable as a technical document
- **Vector Database Integration:** integrate ChromaDB to store previously generated concept narratives and enable semantic search across a library of manufacturing concepts, allowing users to retrieve and compare related outputs
- **Google Imagen via Vertex AI:** replace Pollinations.ai with Google Imagen accessed through Vertex AI for higher-resolution, more controllable image generation at production quality
- **Multiple Image Variations:** generate three to five image variations per concept with different angles, cutaway views, or assembly stages rather than a single static illustration
- **Comparison Mode:** allow the user to enter two manufacturing concepts and view their narratives and images side by side for direct technical comparison
- **Voice Input:** integrate speech-to-text input so users can speak a manufacturing concept rather than type it, improving accessibility and demo fluidity
- **Streamlit Cloud Deployment:** deploy the application to Streamlit Community Cloud with a shared API key pool for public access without requiring users to bring their own Gemini key
- **History Panel:** maintain a session-scoped history of all concepts generated in the current session, allowing the user to scroll back through previous outputs without regenerating

6. References

- **Google Gemini API Documentation:** https://ai.google.dev/gemini-api/docs
- **google-generativeai Python SDK:** https://pypi.org/project/google-generativeai
- **Pollinations.ai:** https://pollinations.ai
- **Flux Image Model by Black Forest Labs:** https://blackforestlabs.ai
- **Streamlit Documentation:** https://docs.streamlit.io
- **Pillow (PIL) Image Library:** https://pillow.readthedocs.io
- **Python urllib.parse Module:** https://docs.python.org/3/library/urllib.parse.html
- **requests HTTP Library:** https://requests.readthedocs.io
- **ChromaDB Vector Database:** https://www.trychroma.com
- **Google Vertex AI (Imagen):** https://cloud.google.com/vertex-ai/generative-ai/docs/image/overview

7. GitHub

The complete source code for the Multimodal Manufacturing Creator is publicly available at:

https://github.com/omtiwari17/Multimodal-Manufacturing-Creator

File	Description
app.py	Complete Streamlit application UI, LLM orchestration, image generation, error handling, and workflow diagram
requirements.txt	Python dependencies: streamlit, google-generativeai, requests, Pillow
README.md	Project overview, setup instructions, example concepts, tech stack, workflow, and group details
.gitignore	Excludes Python cache, .env files, and Streamlit secrets from version control

To run locally: clone the repository, run pip install -r requirements.txt, then streamlit run app.py. Enter your Gemini API key in the sidebar. No other configuration required.